Analyzing the Retrieval–Reasoning Gap in Adaptive RAG Systems*

Anugna Yakkala

School of Computer Science and Engineering,
Vellore Institute of Technology, Vellore, TN, India

April 2026

Abstract

When a 5G base station fails, the root cause usually spans several log files in different formats—and a single retrieval pass often misses part of the chain. We present an iterative RAG pipeline that pairs hybrid retrieval (BM25 + bi-encoder) with cross-encoder reranking and a confidence-gated stopping rule, allowing the system to refine its query and re-retrieve when the initial evidence is insufficient. On a preliminary corpus of ∼30 telecom log records (3 queries), cross-encoder reranking raises Precision@K from ∼0.80 to 0.80–0.85 compared to a dense-only baseline. Root cause accuracy, however, stays flat at ∼0.67 regardless of retrieval method—the frozen LLM (Llama 3.3 70B) appears to be the bottleneck, not the evidence. This result suggests that retrieval quality and reasoning quality degrade for different reasons and benefit from different interventions. All components are publicly available; no domain-specific training is required.

1 Introduction

Modern telecom networks generate logs from diverse subsystems: radio units, control-plane processors, user-equipment emulators, and core network functions. When a failure occurs—for example, a 5G UE context release—the root cause typically involves a chain of events distributed across multiple log files with inconsistent formats and timestamps. An engineer must correlate entries from an eGate console log, a UEC runtime log, and a RAIN system log to trace the sequence from an RRC reconfiguration failure to a bearer cancellation to a UE release.

Retrieval-augmented generation (RAG) [Lewis et al., 2020] is a reasonable starting point: retrieve the log entries most likely to matter, then ask a large language model (LLM) to piece together what happened [Gao et al., 2023]. In practice, though, single-pass RAG runs into two problems here. First, one retrieval pass may miss links in a multi-file causal chain, especially when the initial query is broad (e.g., "Why did UE4 fail?"). Second, bi-encoder similarity search alone may rank lexically-overlapping but causally irrelevant entries above informative ones [Thakur et al., 2021].

We build an adaptive iterative RAG system around three ideas:

1. **Hybrid retrieval** (BM25 + dense bi-encoder) for high-recall candidate generation.
2. **Cross-encoder reranking** [Nogueira and Cho, 2019] that rescores each (query, document) pair via joint cross-attention, improving precision without reducing recall.
3. **Confidence-gated iteration** [Asai et al., 2024, Jiang et al., 2023] where a multi-signal confidence score determines whether to refine the query and re-retrieve, or to return the current result.

*Code available at: https://github.com/yakkalaanugna/Automated-RAG-Agent

The bet, in short, is that iterating on retrieval—with a cross-encoder to keep precision high at each step—can meaningfully improve the evidence an LLM sees, without needing any domain-specific training. Everything in the pipeline is off-the-shelf: a sentence-transformer bi-encoder [Reimers and Gurevych, 2019], an MS MARCO-trained cross-encoder, and a frozen LLM behind an API.

Sections 2–8 cover related work, the method, experimental setup, results, discussion, limitations, and future directions.

2 Related Work

2.1 Retrieval-Augmented Generation

Lewis et al. [2020] showed that augmenting language models with a retrieval step—fetching relevant passages before generation—leads to substantial gains on knowledge-intensive tasks. Follow-up work has pushed RAG into summarization, code generation, and other tasks [Gao et al., 2023]. Most of these systems, though, still follow a single-pass pattern: one query, one retrieval, one generation. The model never goes back to check whether it missed something. For telecom log analysis, where the relevant evidence is scattered across multiple files in different formats, a single pass is often not enough.

2.2 Iterative and Multi-Step Retrieval

A growing line of work tries to move past single-pass retrieval. Self-RAG [Asai et al., 2024] teaches the model itself to decide when retrieval is needed and to critique what it generates; Active RAG [Jiang et al., 2023] uses active learning signals for similar purposes. Separately, query rewriting [Wang et al., 2023] reformulates the search string after an initial pass, which can pull in evidence that the original query missed. What these approaches share, however, is a lack of well-grounded stopping criteria—they tend to either iterate a preset number of times or depend on model-internal confidence that is hard to calibrate in practice. None of them, to our knowledge, have been tested on structured operational logs like those found in telecom networks, where failure chains cut across files with quite different formatting conventions.

2.3 Hybrid Retrieval and Cross-Encoder Reranking

The idea of mixing sparse and dense retrieval is by now well established: BM25 [Robertson and Zaragoza, 2009] handles exact term matching, while bi-encoder models [Reimers and Gurevych, 2019] capture semantic overlap, and fusing the two gives better recall than either alone. On top of this, cross-encoder rerankers [Nogueira and Cho, 2019]—typically trained on MS MARCO [Nguyen et al., 2016]—can substantially sharpen precision by scoring each (query, passage) pair through joint cross-attention. BEIR [Thakur et al., 2021] has confirmed that these models generalize reasonably well zero-shot, and late-interaction designs like ColBERT [Khattab and Zaharia, 2020] trade off some accuracy for speed. What is less explored is using rerankers inside an iterative loop. In practice, cross-encoders are almost always applied once, after a single retrieval pass. There is little work on re-applying them after query refinement, where the reranker could catch different relevant documents each time.

2.4 Positioning of This Work

Telecom log root cause analysis sits at an awkward intersection: it needs multi-step reasoning across files with inconsistent formats, but it also needs precision—pulling in irrelevant log lines wastes the LLM's limited context window. We are not aware of existing work that combines hybrid retrieval, cross-encoder reranking, and confidence-based stopping in an iterative loop for this kind of structured operational data. Our system targets that gap.

3 Method

The system has five stages arranged in a conditional loop. In short:

`Query` → `Hybrid Retrieval` → `Cross-Encoder Reranking` → `Top-`k `Selection` → `LLM Reasoning` → `Confidence` → `Iterate?`

Figure 1 expands this into more detail.

```
Query
  |
  v
Log Parsing  -->  Document Construction
  |
  v
[LOOP (max M iterations)]
  |
  v
Hybrid Retrieval (BM25 + Dense)    ... recall
  |
  v
Cross-Encoder Reranking            ... precision
  |
  v
Top-k Selection  -->  LLM Analysis ... reasoning
  |
  v
Confidence Scoring
  |
  +---> High confidence --> Return best result
  |
  +---> Low confidence  --> Refine query, iterate
```

Figure 1: Pipeline overview. Hybrid retrieval maximizes *recall*, cross-encoder reranking maximizes *precision*, LLM analysis performs *reasoning*, and iteration provides *refinement*.

3.1 Log Parsing and Document Construction

Raw telecom logs arrive in vendor-specific formats. Our parser handles eGate console logs, UEC runtime logs, RAIN system logs, syslog/btslog, and compressed archives. Each line is parsed into a structured record: severity, source filename, line number, timestamp, module name, error code, and raw message.

Parsed records are converted to documents with natural-language content (e.g., "ERROR log1.txt line 4 at 18:34:08 UEC-1 code 4 ...") rather than bracket-formatted prefixes. This improves both bi-encoder embedding quality (the model was trained on natural text) and BM25 matching (terms like "error" tokenize cleanly). Structured metadata is preserved separately for display.

3.2 Hybrid Candidate Generation (Stage 1: Recall)

The first stage maximizes recall by fusing two complementary retrieval signals.

Dense retrieval. Documents are embedded with `all-MiniLM-L6-v2` [Reimers and Gurevych, 2019] (22M params, 384-dim, normalized) and indexed in ChromaDB. Top-N candidates are

retrieved by cosine similarity. Late-interaction models like ColBERT [Khattab and Zaharia, 2020] might do better here, but we opt for the simpler bi-encoder to keep the pipeline lightweight.

BM25 retrieval. A parallel Okapi BM25 index [Robertson and Zaragoza, 2009] ($k_1 = 1.5$, $b = 0.75$) captures exact lexical matches—particularly effective for error codes and identifiers that dense models may under-weight.

Score fusion. Results are deduplicated by content and combined:

$$s_{\text{hybrid}}(d) = \alpha \cdot s_{\text{dense}}(d) + (1 - \alpha) \cdot s_{\text{BM25}}(d) \tag{1}$$

with $\alpha = 0.7$. BM25 scores are min-max normalized to $[0, 1]$. This stage deliberately over-fetches ($N = 20$–30) to provide a high-recall candidate pool for reranking.

3.3 Cross-Encoder Reranking (Stage 2: Precision)

The candidate set is rescored by a cross-encoder (`ms-marco-MiniLM-L-6-v2`, 22M params) [Nogueira and Cho, 2019]. Because the cross-encoder reads the query and document jointly rather than encoding them separately, it can pick up on token-level interactions that bi-encoders miss. Raw logits are sigmoid-normalized:

$$s_{\text{CE}}(q, d) = \sigma\big(\text{CE}(q, d)\big) = \frac{1}{1 + e^{-\text{CE}(q,d)}} \tag{2}$$

The cross-encoder score is the *sole* ranking signal after Stage 1, following the retrieve-then-rerank paradigm evaluated on MS MARCO [Nguyen et al., 2016]. We deliberately discard the hybrid fusion score: the cross-encoder already captures both lexical and semantic relevance through joint encoding, and mixing in the Stage 1 score would reintroduce the noise reranking is designed to eliminate.

The top-k documents (default $k = 6$) are forwarded to the LLM.

3.4 LLM Analysis and Confidence Scoring (Stage 3: Reasoning)

The top-k reranked documents are injected into a structured prompt for the LLM (Llama 3.3 70B via Groq, temperature 0). The prompt requests: root cause summary, severity, chronological error timeline with file/line/timestamp references, failure chain explanation, step-by-step reasoning, and a remediation recommendation.

The output is scored on four signals to produce a composite confidence:

1. **Retrieval quality** — mean cross-encoder score of retrieved documents.
2. **Completeness** — presence of all required output sections.
3. **Evidence density** — count of specific file/line/timestamp references, normalized.
4. **Consistency** — analysis overlap with the previous iteration (higher indicates stable conclusions).

3.5 Confidence-Gated Iteration (Stage 4: Refinement)

The confidence score determines whether to iterate or return. Three stopping rules apply:

1. **High-confidence exit**: confidence ≥ 0.85.
2. **Plateau detection**: confidence fails to improve by $\geq \epsilon = 0.01$ for two consecutive iterations.
3. **Hard cap**: $M = 3$ iterations.

One confidence dip is tolerated, since query narrowing can temporarily reduce coverage before finding better evidence. The system returns the *best-scoring* iteration's result, not necessarily the last.

Algorithm 1 Adaptive Iterative RAG

Require: Query q, documents $\mathcal{D}$, max iterations M, threshold ϵ

$q_1 \leftarrow q$; $c_0 \leftarrow 0$; best $\leftarrow$ NULL; seen $\leftarrow \emptyset$
for $i = 1$ **to** M **do**
  $\mathcal{C}_i \leftarrow \text{HybridRetrieve}(q_i, \mathcal{D}, N_i)$ ▷ Stage 1: BM25 + Dense
  $\mathcal{R}_i \leftarrow \text{Rerank}(q_i, \mathcal{C}_i, k)$ ▷ Stage 2: Cross-encoder
  $\mathcal{R}_i \leftarrow \text{Dedup}(\mathcal{R}_i, \text{seen})$; update seen
  $a_i \leftarrow \text{LLM}(q_i, \mathcal{R}_i)$ ▷ Structured analysis
  $c_i \leftarrow \text{Confidence}(a_i, \mathcal{R}_i, a_{i-1})$
  if $c_i > c_{\text{best}}$ **then**
    best $\leftarrow (a_i, \mathcal{R}_i, c_i)$
  end if
  if $c_i \geq 0.85$ **or** consecutive drops ≥ 2 **then**
    break
  end if
  $q_{i+1} \leftarrow \text{RefineQuery}(q, q_i, a_i)$
end for
return best

3.6 Query Refinement

When iteration continues, the LLM rewrites the query [Wang et al., 2023] using discovered error codes, module names, timestamps, and identified gaps. For example, "network failure" may become "RRC Reconfiguration failure code 4 rfma_impl.cpp UE4 18:34:08". On iteration $i > 1$, the candidate pool expands from 20 to 30, and previously-seen documents are deprioritized to surface novel evidence.

Algorithm 1 summarizes the full pipeline.

4 Experimental Setup

4.1 Dataset

We evaluate on a small corpus of telecom log files collected from a 5G gNB test environment. The corpus consists of 3 log files (eGate console, UEC runtime, RAIN system) totaling approximately 30 parsed records. The logs capture a real failure scenario involving an RRC reconfiguration failure, bearer cancellation, and UE context release distributed across the three files.

We define 3 evaluation queries with manually annotated ground truth:

1. "Why did UE4 fail?" — root cause: RRC Reconfiguration failure code 4 in `rfma_impl.cpp`
2. "What caused the UE context release?" — root cause: CU-CP-initiated release following RRC failure
3. "Why were packets lost for UE4?" — root cause: forward jump after UE4 RRC failure and release

Ground truth includes relevant document identifiers and root cause keywords for metric computation.

4.2 Compared Methods

We compare three system configurations, all sharing the same LLM and log corpus:

- **Baseline RAG**: Single-pass dense-only retrieval (no BM25, no reranking, no iteration). Top-k documents are passed directly to the LLM.
- **Fixed Iterative RAG**: Hybrid retrieval (BM25 + dense) without cross-encoder reranking, iterated 3 times with query refinement regardless of confidence.

- **Adaptive Iterative RAG** (ours): Full pipeline with hybrid retrieval, cross-encoder reranking, and confidence-gated iteration (max 3).

4.3 Metrics

- **Precision@K**: Fraction of the top-k retrieved documents that are relevant (matched by source file identifier).
- **Recall@K**: Fraction of relevant documents appearing in the top-k results.
- **Root Cause Accuracy**: Fractional keyword overlap between the LLM's predicted root cause and the ground truth keyword set. Computed as |matched keywords| / |total keywords|.
- **Latency**: Wall-clock time for the full pipeline.
- **Confidence Trajectory**: Per-iteration confidence score to visualize convergence behavior.

4.4 Implementation Details

- **Bi-encoder**: `all-MiniLM-L6-v2` (22M params, 384-dim embeddings, normalized)
- **Cross-encoder**: `ms-marco-MiniLM-L-6-v2` (22M params, max sequence length 256)
- **LLM**: Llama 3.3 70B via Groq API (temperature = 0, max tokens = 2048)
- **Vector store**: ChromaDB (in-memory ephemeral client)
- **Hybrid weight**: $\alpha = 0.7$ (dense), $1 - \alpha = 0.3$ (BM25)
- **Top-k**: 6 documents per iteration
- **Candidate pool**: 20 (iteration 1), 30 (subsequent)
- **Max iterations**: 3
- **Convergence threshold**: $\epsilon = 0.01$

All experiments run on CPU. The cross-encoder adds approximately 40–100 ms per iteration for 20–30 candidates, which is negligible compared to the LLM inference latency of approximately 8–12 s per call.

5 Results

Table 1 presents preliminary results averaged across 3 queries. We emphasize that these are based on a small dataset and should not be interpreted as statistically significant.

Table 1: Preliminary comparison on 3 telecom log queries (∼30 records). Results averaged across queries. Not statistically significant.

Method	Precision@K	Root Cause Acc.	Avg. Iters	Latency
Baseline RAG	∼0.80	∼0.67	1	∼2.5 s
Fixed Iterative	∼0.69	∼0.67	3	∼20 s
Adaptive (ours)	0.80–0.85	∼0.67	2.0	∼16 s

5.1 Key Observations

Four patterns stand out:

- **Cross-encoder reranking improves Precision@K.** The adaptive method achieves the highest Precision@K (0.80–0.85), suggesting that the cross-encoder effectively promotes causally relevant documents above lexically similar but uninformative ones. A similar pattern has been reported for open-domain QA [Nogueira and Cho, 2019].
- **Iteration without reranking can *degrade* precision.** The fixed iterative method (∼0.69) scores lower than the single-pass baseline (∼0.80). This indicates that query

refinement without a precision-oriented reranking stage may introduce drift, retrieving tangentially related but irrelevant entries on subsequent passes.

- **Root cause accuracy does not improve with better retrieval.** All three methods achieve ∼0.67, despite differences in retrieval quality. Because the same frozen LLM (Llama 3.3 70B, temperature 0) is used throughout, reasoning capability—not evidence quality—appears to be the binding constraint. The LLM, not the retriever, appears to be the ceiling here.
- **Adaptive iteration helps on underspecified queries.** For the broad query "Why did UE4 fail?", the adaptive system converged in 2 iterations (confidence: 0.71 → 0.76), with the second pass using a refined query containing specific error codes from the first pass. The confidence-gated mechanism avoided a wasteful third iteration.

5.2 Latency

The adaptive method incurs ∼16 s versus ∼2.5 s for the baseline. The cross-encoder reranking step itself is fast (∼50–100 ms per iteration); latency is dominated by additional LLM calls (∼8–12 s each). The adaptive system averages 2 iterations versus the fixed method's 3, saving one LLM call.

5.3 Summary of Takeaways

To distill the results concisely:

- Cross-encoder reranking is the primary driver of Precision@K gains—it promotes genuinely relevant evidence over surface-level lexical matches.
- Iterative query refinement helps most when the initial query is vague; on already-specific queries, the first pass often suffices.
- Root cause accuracy remains flat (∼0.67) regardless of retrieval method—the frozen LLM caps reasoning quality independently of evidence quality.
- Retrieval and reasoning seem to be limited by different factors—fixing one does not fix the other.

6 Discussion

6.1 The Retrieval–Reasoning Gap

If there is one thing these experiments make clear, it is this: **better retrieval does not, on its own, produce better reasoning when the language model stays the same.** All three methods land at essentially the same root cause accuracy (∼0.67), even though Precision@K varies noticeably. Retrieval and reasoning, it turns out, fail for different reasons—and improving one does not automatically fix the other. End-to-end RAG evaluations tend to blur this distinction.

We attribute this gap to two factors:

1. **LLM ceiling**: The frozen Llama 3.3 70B model's reasoning is bounded by its training data and architecture, regardless of input quality.
2. **Sufficient baseline context**: On a small corpus, even single-pass retrieval surfaces enough relevant entries for the LLM to identify the root cause. The marginal value of additional relevant documents diminishes rapidly.

On a larger corpus, where single-pass retrieval is more likely to miss links in the causal chain, we would expect the retrieval gap between methods to widen—but whether the reasoning gap follows is an open question that depends on the LLM.

6.2 Why Cross-Encoder Reranking Improves Precision

BM25 is good at exact term matches—error codes, module names—but blind to paraphrases. The bi-encoder handles semantic similarity yet tends to under-weight rare domain identifiers. Fusing the two gives high recall, though the ranking is still noisy.

The cross-encoder resolves this by jointly encoding the query and each candidate through full cross-attention, which lets it notice, for example, that "failure code 4" in a query is highly relevant to "Failure (code 4) while applying RRCReconfiguration" despite surface-form differences. Our results suggest that this reranking accounts for most of the Precision@K improvement, consistent with Nogueira and Cho [2019].

6.3 Adaptive versus Fixed Iteration

Adaptive stopping helps in two ways: it skips LLM calls that would not improve the answer, and by returning the *best-scoring* iteration rather than the last, it tolerates the temporary confidence dips that query narrowing can cause. The fixed iterative method lacks both properties and achieves lower Precision@K than the baseline, indicating that blind iteration can cause query drift.

7 Limitations

This work has several clear limitations:

1. **Small dataset**. Our evaluation uses only 3 queries over ∼30 log records. The results are preliminary and not statistically significant. We cannot claim generalizability to larger or more diverse telecom environments.
2. **No ablation study**. We have not isolated the individual contributions of BM25 fusion, cross-encoder reranking, and iterative refinement through a full factorial ablation. Our discussion of which component drives precision gains is based on qualitative observation, not controlled experiments.
3. **Single LLM**. All experiments use a single frozen LLM (Llama 3.3 70B). Results may differ with other models, particularly those with stronger or weaker reasoning capabilities.
4. **Latency**. The system adds approximately 13 s of overhead compared to single-pass RAG, primarily from additional LLM calls. This may be unacceptable for real-time monitoring applications.
5. **No domain adaptation**. Both the bi-encoder and cross-encoder are general-purpose models trained on web text and MS MARCO passages. Telecom-specific vocabulary (e.g., "NGAP", "CU-CP", "gNB") is out-of-distribution. Domain-adapted models could improve both retrieval and reranking quality.
6. **Confidence calibration**. The composite confidence score is a heuristic combination of four signals. It has not been calibrated against human judgments of analysis quality, so the stopping thresholds may not be optimal.

8 Future Work

We see several natural next steps:

- **Larger evaluation corpora** with statistical significance testing (e.g., bootstrap confidence intervals over 50+ queries from diverse failure scenarios).
- **Full ablation study** varying each component (± BM25, ± reranker, ± iteration) to quantify individual contributions.
- **Domain-adapted embeddings** fine-tuned on telecom vocabulary using contrastive learning on (query, relevant log) pairs.

- **Latency optimization** through candidate pruning, cross-encoder distillation, or batched LLM inference.
- **Graph-based retrieval** that models explicit causal relationships between log events, enabling structural traversal rather than purely similarity-based retrieval.
- **Multi-LLM comparison** to disentangle retrieval effects from reasoning effects across models of different capabilities.
- **Human evaluation** of root cause quality, since keyword-based accuracy metrics may not capture the full informativeness of a diagnosis.

9 Conclusion

We built an iterative RAG pipeline for telecom log root cause analysis and tested it on a small 5G gNB corpus. Cross-encoder reranking was the main source of retrieval precision gains; adaptive stopping cut unnecessary LLM calls without sacrificing quality. The most striking outcome was negative: despite measurably better retrieval, root cause accuracy did not budge—the frozen LLM was the limiting factor, not the evidence. For practitioners, the implication is that retrieval and reasoning should be profiled and improved separately. All models used are publicly available and require no domain-specific fine-tuning.

References

Asai, A., Wu, Z., Wang, Y., Sil, A., and Hajishirzi, H. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In *ICLR*, 2024.

Gao, Y., Xiong, Y., Dibia, V., et al. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*, 2023.

Jiang, Z., Xu, F. F., Gao, L., et al. Active retrieval augmented generation. In *EMNLP*, 2023.

Khattab, O. and Zaharia, M. ColBERT: Efficient and effective passage search via contextualized late interaction over BERT. In *SIGIR*, 2020.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., and Kiela, D. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *NeurIPS*, 2020.

Nguyen, T., Rosenberg, M., Song, X., Gao, J., Tiwary, S., Majumder, R., and Deng, L. MS MARCO: A human generated machine reading comprehension dataset. In *NeurIPS Workshop on Cognitive Computation*, 2016.

Nogueira, R. and Cho, K. Passage re-ranking with BERT. *arXiv preprint arXiv:1901.04085*, 2019.

Reimers, N. and Gurevych, I. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *EMNLP*, 2019.

Robertson, S. and Zaragoza, H. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.

Thakur, N., Reimers, N., Rücklé, A., Srivastava, A., and Gurevych, I. BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models. In *NeurIPS Datasets and Benchmarks*, 2021.

Wang, L., Yang, N., and Wei, F. Query2doc: Query expansion with large language models. In *EMNLP*, 2023.